

RCA Implementation Plan — v2.0

ESGA DataPower Root Cause Analyzer Chatbot

Version: 2.0

Date: 2026-06-04

Supersedes: RCA_Implementation_Plan.pdf (v1.0, 2026-05-26)

Changes incorporated from:

- RCA_Stack_Update_Supabase_Vercel.md (2026-05-31) — full stack replacement
- RCA_Phase1_Updated_Spec.md (2026-05-31) — ingestion pipeline detail
- RCA_Evaluation_Strategy.md (2026-05-31) — Phase 7 evaluation strategy

Project Overview

AI-powered incident troubleshooting chatbot for ESGA DataPower support engineers. Retrieves relevant runbook sections from the ESGA knowledge base and generates grounded, cited recommendations for diagnosing and resolving DataPower incidents.

Knowledge base: ESGA DataPower runbooks (Markdown), covering 10 failure patterns (Patterns 1–10).

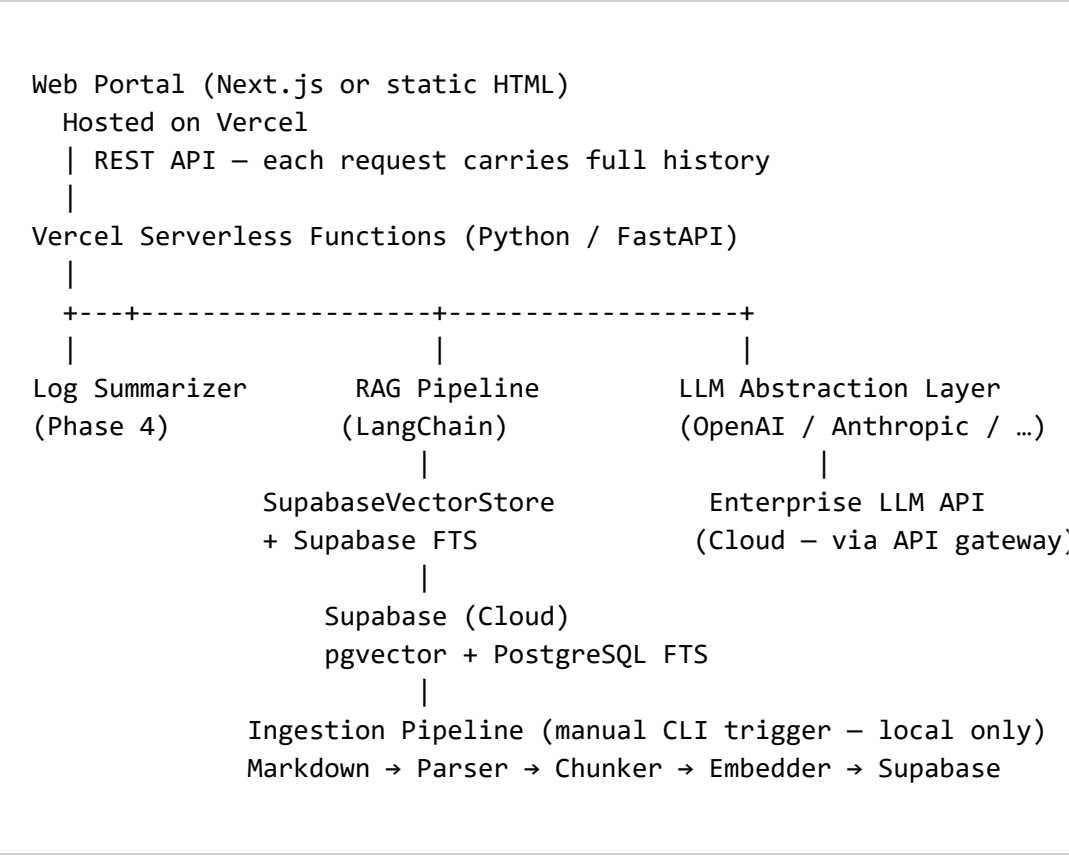
Primary users: ESGA support engineers triaging live DataPower incidents.

Stack Summary

Component	v1.0 (Original)	v2.0 (Current)
Document parser	python-docx	markdown-it-py
Vector store	ChromaDB (local, on-prem)	Supabase pgvector (cloud, managed)
Sparse index	rank_bm25 library + disk file	Supabase native FTS (tsvector)
LangChain vector store class	Chroma	SupabaseVectorStore
Deployment — frontend	On-prem app server	Vercel

Deployment — backend API	On-prem app server (Docker)	Vercel Serverless Functions (Python)
Environment config	Server env vars / .env	Vercel Environment Variables dashboard
Infrastructure maintenance	Self-managed	Supabase managed; Vercel managed

System Architecture



Project Structure

root/	
├─ api/	← Vercel Python serverless functions
│ └─ analyze.py	← POST /api/analyze
│ └─ health.py	← GET /api/health
├─ frontend/	← Static web portal (HTML/JS or Next.js)
│ └─ index.html	
├─ source/	← ESGA runbook Markdown files (RAG knowledge)
├─ eval/	← Evaluation scripts and test set
│ └─ golden_test_set.csv	
│ └─ run_retrieval_eval.py	
│ └─ run_generation_eval.py	
│ └─ report.py	
│ └─ results/	
├─ vercel.json	← Routing + function config
├─ requirements.txt	← Python dependencies (production)
└─ ingest.py	← Local CLI only – not deployed to Vercel

Phase 1 — Ingestion Pipeline

Overview

The ingestion pipeline runs locally as a CLI command. It parses ESGA runbook Markdown files, splits them into chunks at defined heading boundaries, enriches each chunk with metadata, prepends a context header for embedding, and upserts to Supabase.

```
python ingest.py --docs ./source/
```

1.1 Document Parser — `markdown-it-py`

Source files are `.md` (Markdown). The original `python-docx` parser is replaced.

```
from markdown_it import MarkdownIt

md = MarkdownIt()
tokens = md.parse(raw_text)
```

The parser must:

- Extract heading levels (`#`, `##`, `###`) to drive chunk boundaries
- Preserve table content as plain text (convert pipe-table rows to readable strings)

- Preserve fenced code blocks (Splunk queries, bash commands, JavaScript snippets) intact — never split mid-block
- Retain heading text as the `section_title` metadata field

1.2 Chunking Boundaries

Heading Level	Section	Rule
##	Sections 1, 2, 4, 5 (parent), 6, 7	One chunk per ## section
###	Section 5 sub-sections (5.1, 5.2, 5.3, 5.4)	One chunk per ### sub-section

Section 5 combined is 1,500–2,000+ tokens across 3–4 sub-sections. Each sub-section covers a distinct failure scenario with its own symptoms, resolution steps, and validation commands. Splitting at ### preserves retrieval precision — a query about a timeout config issue retrieves section 5.2, not the entire troubleshooting block.

Sections excluded from indexing (low retrieval value, add noise):

Section	Content	Action
Section 3 — Architecture Context	ASCII art diagrams	Excluded
Section 8 — Related Runbooks	Cross-reference links	Excluded (captured in <code>related_patterns</code> metadata)
Section 9 — Revision History	Version/date table	Excluded

1.3 Chunk Header Injection

Each chunk has a plain-text context header prepended **before embedding only**. The header is not stored in the content column — only the clean chunk body is stored and returned to the LLM.

Format:

```
[ESGA {pattern_name} | {section_title}]
```

```
{chunk_body}
```

Example:

****Symptoms:**** Specific backend times out intermittently under load.
peak traffic windows. Backend team confirms processing time exceeds
...

1.4 Metadata Schema

Full metadata stored per chunk in Supabase jsonb:

```
{
  # Original fields
  "runbook_name": str, # e.g. "ESGA_Pattern_1_Backend_Connectivity_Troubleshooting: 5.2 Timeout"
  "section_title": str, # e.g. "Troubleshooting: 5.2 Timeout"
  "pattern_id": str, # e.g. "Pattern_1"
  "chunk_index": int, # sequential index within the document

  # Added fields
  "pattern_name": str, # e.g. "Backend Connection Timeout"
  "category": str, # e.g. "Backend Connectivity"
  "severity": str, # e.g. "P2 - High"
  "section_type": str, # one of: overview | error_signature | troubleshooting | mistake
  "error_codes": list, # e.g. ["0x00d30003", "ETIMEDOUT",
  "http_status_codes": list, # e.g. ["503", "404"]
  "related_patterns": list, # e.g. ["Pattern_2", "Pattern_9", "Pattern_10"]
}
```

Extraction logic (no LLM required):

Field	Source
pattern_name	Document # Title heading, strip "ESGA Pattern N Runbook: " prefix
category	Document Information table, "Category" row
severity	Document Information table, "Severity" row
section_type	Map heading text → enum at parse time (see mapping below)
error_codes	Regex scan: `0x[0-9A-Fa-f]+`, `[A-Z_]{5,}` in error tables
http_status_codes	Regex scan: HTTP [0-9]{3} or `[0-9]{3}` adjacent to status label

related_patterns Section 8 link list, extract Pattern_N from each link

Section type mapping:

```
SECTION_TYPE_MAP = {
    "overview": "overview",
    "error signatures": "error_signatures",
    "triage decision tree": "triage",
    "troubleshooting steps": "troubleshooting", # parent – not
    "common mistakes to avoid": "mistakes",
    "escalation matrix": "escalation",
}
# Section 5.x sub-sections inherit section_type = "troubleshooting"
```

1.5 Supabase Schema

Run once in the Supabase SQL editor before first ingestion.

```
-- Enable pgvector
create extension if not exists vector;

-- Main documents table
create table documents (
    id          bigserial primary key,
    content     text        not null,           -- clean chunk body (no
    metadata    jsonb       not null default '{}',
    embedding   vector(1536)           -- text-embedding-3-small
);

-- Full-text search column (auto-maintained by Postgres)
alter table documents
    add column fts tsvector
    generated always as (to_tsvector('english', content)) stored;

-- Indexes
create index on documents using ivfflat (embedding vector_cosine_ops)
    with (lists = 100);
create index on documents using gin(fts);
create index on documents using gin(metadata);
```

match_documents RPC — required by LangChain SupabaseVectorStore:

```

create or replace function match_documents (
    query_embedding vector(1536),
    match_count     int      default 10,
    filter          jsonb    default '{}'
)
returns table (
    id          bigint,
    content     text,
    metadata    jsonb,
    similarity  float
)
language plpgsql
as $$
begin
    return query
    select
        d.id,
        d.content,
        d.metadata,
        1 - (d.embedding <=> query_embedding) as similarity
    from documents d
    where d.metadata @> filter
    order by d.embedding <=> query_embedding
    limit match_count;
end;
$$;

```

1.6 Ingestion Script

```

# requirements – replace chromadb with supabase
# REMOVE: chromadb, rank_bm25
# ADD:      supabase, langchain-community

from supabase import create_client
from langchain_community.vectorstores import SupabaseVectorStore
from langchain_openai import OpenAIEmbeddings

supabase = create_client(
    supabase_url=os.environ["SUPABASE_URL"],
    supabase_key=os.environ["SUPABASE_SERVICE_KEY"] # service role key
)

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = SupabaseVectorStore(
    client=supabase,
    embedding=embeddings,
    table_name="documents",
    query_name="match_documents",
)

# Ingest – header prepended for embedding, clean body stored in content
vectorstore.add_texts(
    texts=[chunk.embed_text for chunk in chunks], # [header + body]
    metadatas=[chunk.metadata for chunk in chunks],
    ids=[chunk.id for chunk in chunks],
)

```

1.7 Environment Variables (Ingestion — Local Only)

```

SUPABASE_URL=https://<project-id>.supabase.co
SUPABASE_SERVICE_KEY=<service-role-key> # write access; never deploy
OPENAI_API_KEY=<key>

```

1.8 Phase 1 Deliverables

#	Deliverable
1	ingest.py — ingestion script using markdown-it-py
2	Supabase documents table populated with enriched metadata
3	Supabase FTS index (tsvector) configured for sparse retrieval
4	Section exclusion list applied at parse time (Sections 3, 8, 9)
5	Chunk header injection applied at embedding time

Phase 2 — Core RAG (Single-Turn)

Overview

Basic RAG chain: user query → dense vector retrieval → LLM generation with citations.
Establishes the baseline for evaluation Gate A.

Retriever

```
from langchain_community.vectorstores import SupabaseVectorStore

retriever = SupabaseVectorStore(
    client=supabase,
    embedding=embeddings,
    table_name="documents",
    query_name="match_documents",
).as_retriever(search_kwargs={"k": 5})
```

Citation Format

Each answer must cite the source chunk using the format:

```
[Pattern_1 | Error Signatures]
```

API Contract

```
POST /api/analyze
{
  "query": str,
  "history": list[{role: str, content: str}] # full history sent to LLM
}

Response:
{
  "answer": str,
  "sources": list[{pattern_id: str, section_title: str}]
}
```

Phase 2 Deliverables

#	Deliverable
1	RAG chain with SupabaseVectorStore retriever
2	LLM abstraction layer (model-agnostic: OpenAI / Anthropic / Azure OpenAI)
3	FastAPI endpoint POST /api/analyze
4	GET /api/health health check endpoint
5	Evaluation Gate A passed (see Phase 7)

Phase 3 — Advanced RAG (Hybrid Retrieval)

Overview

Extends Phase 2 with HyDE query expansion, hybrid dense + sparse retrieval, and Reciprocal Rank Fusion (RRF). Improves error-code routing accuracy by combining semantic and keyword signals.

Dense Retrieval (Supabase pgvector)

```
dense_results = vectorstore.similarity_search_with_score(
    query=hyde_passage,
    k=10,
    filter=metadata_filter    # optional jsonb pre-filter
)
```

Sparse Retrieval (Supabase FTS — replaces rank_bm25)

```
def sparse_search(query: str, k: int = 10, filter: dict = None) -> List[Document]:
    ts_query = " | ".join(query.split())

    rpc_params = {"query_text": ts_query, "match_count": k}
    if filter:
        rpc_params["filter"] = filter

    return supabase.rpc("match_documents_fts", rpc_params).execute()
```

Additional RPC function (run once in Supabase):

```

create or replace function match_documents_fts (
    query_text    text,
    match_count   int    default 10,
    filter        jsonb default '{}'
)
returns table (
    id          bigint,
    content     text,
    metadata    jsonb,
    rank        float
)
language plpgsql
as $$
begin
    return query
    select
        d.id,
        d.content,
        d.metadata,
        ts_rank(d.fts, to_tsquery('english', query_text))::float as rank
    from documents d
    where d.fts @@ to_tsquery('english', query_text)
        and d.metadata @> filter
    order by rank desc
    limit match_count;
end;
$$;

```

Metadata Pre-Filtering Examples

```

# Error code query – pre-filter before vector search
vectorstore.similarity_search(
    query=hyde_query,
    k=5,
    filter={"error_codes": "JWT_SIGNATURE_INVALID"}
)

# Escalation query – only return escalation matrix chunks
vectorstore.similarity_search(
    query=hyde_query,
    k=3,
    filter={"section_type": "escalation"}
)

```

RRF Merge

```
def reciprocal_rank_fusion(
    dense_results: list,
    sparse_results: list,
    k: int = 60
) -> list:
    scores = {}
    for rank, doc in enumerate(dense_results):
        scores[doc.id] = scores.get(doc.id, 0) + 1 / (k + rank + 1)
    for rank, doc in enumerate(sparse_results):
        scores[doc.id] = scores.get(doc.id, 0) + 1 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Phase 3 Deliverables

#	Deliverable
1	HyDE query expansion integrated
2	Sparse retrieval via Supabase FTS (match_documents_fts RPC)
3	RRF merge of dense + sparse results
4	Metadata pre-filtering for error code and escalation queries
5	Evaluation Gate B passed (see Phase 7)

Phase 4 — Splunk Log Integration

Stack impact: None. Log summarizer component is backend-only and not affected by the vector store or deployment platform change.

Overview

Operators paste raw Splunk log output into the chatbot. A log summarizer component extracts the key signals (error codes, timestamps, affected services) and prepends a structured summary to the RAG query context before retrieval.

Phase 4 Deliverables

#	Deliverable
1	Log summarizer component (LLM-assisted extraction)
2	Structured log summary injected into retrieval context
3	API contract updated to accept raw log text alongside user query

Phase 5 — Multi-Turn Conversation

Stack impact: None. Stateless conversation design, history window management, and context-aware retrieval are not affected by the stack change.

Overview

Conversation history is managed entirely client-side. Each API request carries the full history window. The server is stateless — no session storage.

History Window

The HISTORY_WINDOW environment variable controls the maximum number of turns included in each request (default: 10). Older turns are dropped client-side.

Context-Aware Retrieval

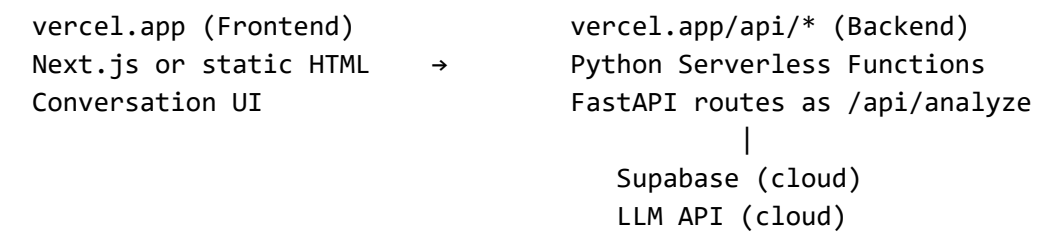
The RAG query is reformulated using recent conversation history to resolve pronoun references and maintain topic continuity across turns.

Phase 5 Deliverables

#	Deliverable
1	Client-side history management with configurable window
2	History-aware query reformulation before retrieval
3	Updated API contract (history field validated)

Phase 6 — Hardening & Production (Vercel)

Architecture



Both frontend and backend deploy from the same Git repository. Vercel auto-deploys on push to main.

vercel.json

```
{
  "functions": {
    "api/*.py": {
      "runtime": "vercel-python@3.x",
      "maxDuration": 60
    }
  },
  "routes": [
    { "src": "/api/(.*)", "dest": "/api/$1" },
    { "src": "/(.*)", "dest": "/frontend/$1" }
  ]
}
```

maxDuration: 60 seconds requires **Vercel Pro** plan. LLM API calls routinely exceed the 10-second Hobby plan limit — Pro is required for production.

Environment Variables (Vercel Dashboard)

Set via **Vercel Project** → **Settings** → **Environment Variables**. Never commit secrets to the repository.

Variable	Description	Used By
SUPABASE_URL	Supabase project URL	RAG pipeline, retriever
SUPABASE_ANON_KEY	Public anon key (read-only queries)	API serverless functions
OPENAI_API_KEY	OpenAI API key	Embeddings + LLM calls
MODEL_PROVIDER	openai / anthropic / azure_openai	LLM abstraction layer
LLM_MODEL	e.g. gpt-4o, claude-opus-4-8	LLM abstraction layer
HISTORY_WINDOW	Max conversation turns (default: 10)	Phase 5 conversation

Use SUPABASE_ANON_KEY (not service role key) in deployed functions — read-only access is sufficient for query-time retrieval. SUPABASE_SERVICE_KEY stays local (ingestion only).

Ingestion is Local Only

`ingest.py` runs locally against Supabase cloud and is never deployed to Vercel. This keeps the `SUPABASE_SERVICE_KEY` off Vercel entirely.

Phase 6 Deliverables

#	Deliverable
1	<code>vercel.json</code> routing and function config
2	<code>api/analyze.py</code> and <code>api/health.py</code> serverless functions
3	Frontend deployed on Vercel
4	All environment variables set in Vercel dashboard
5	Evaluation Gate C passed (see Phase 7)

Phase 7 — RAG Pipeline Evaluation

Overview

Evaluation is a standalone phase run **locally after each milestone** — never part of the Vercel deployment pipeline. Results are stored in Supabase `eval_runs` table for cross-phase comparison.

7.1 Evaluation Goals

Question	Dimension	When
Are the right chunks being retrieved?	Retrieval quality	After Phase 2, 3
Is the generated answer grounded and correct?	Generation quality	After Phase 2, 3, 6
Are error codes and citations routed correctly?	System correctness	After Phase 3, 6

7.2 Golden Test Set

57 test cases — 52 positive and 5 negative. Built once before Phase 2 begins and reused across all phases.

File: `eval/golden_test_set.csv`

Distribution: Pattern 1 (6), Pattern 2 (5), Pattern 3 (6), Pattern 4 (5), Pattern 5 (5), Pattern 6 (5), Pattern 8 (5), Pattern 9 (5), Pattern 10 (5), Negatives (5).

Query Types

Type	Description	Example
error_code	Query contains an exact error code or HTTP status	"DataPower is returning ETIMEDOUT when connecting to a backend"
symptom	Observed behaviour described in plain language	"Multiple unrelated backend services started failing with timeouts at the same time"
trriage	How to start diagnosing a class of error	"A backend connection timeout has occurred. How do I determine the root cause?"
procedure	How to perform a specific action	"How do I flush the DNS cache on DataPower?"
escalation	Who to contact or what the SLA is	"A backend timeout issue has persisted for over 30 minutes. Who do I escalate to?"
negative	No relevant runbook exists	"How do I configure a new Multi-Protocol Gateway service on DataPower from scratch?"

CSV Schema

id, query, query_type, expected_pattern_ids, expected_section_types,

List fields use | as the in-cell delimiter. Negative cases have empty list fields and a ground_truth_answer of None.

Loading the Test Set


```

import csv

def load_golden_test_set(path: str = "eval/golden_test_set.csv") ->
    with open(path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        cases = []
        for row in reader:
            cases.append({
                "id": row["id"],
                "query": row["query"],
                "query_type": row["query_type"],
                "expected_pattern_ids": [p for p in row["expected_pattern_ids"]],
                "expected_section_types": [s for s in row["expected_section_types"]],
                "expected_section_titles": [t for t in row["expected_section_titles"]],
                "ground_truth_answer": row["ground_truth_answer"]
            })
        return cases

```

7.3 Metrics

Retrieval Metrics (no LLM required — fast)

Metric	Definition	Target
Hit Rate @ 5 (HR@5)	Fraction of queries where at least one expected chunk is in top-5 results	≥ 0.85
Mean Reciprocal Rank (MRR@5)	Average of 1/rank for the first relevant result	≥ 0.75
Context Precision @ 5	Of 5 retrieved chunks, fraction that are relevant	≥ 0.70
Context Recall	Of all expected chunks, fraction that appear in top-5	≥ 0.80
Error Code Routing Accuracy	For error_code queries, error_signatures chunk ranks in top-3	≥ 0.90
Escalation Routing Accuracy	For escalation queries, escalation chunk ranks in top-3	≥ 0.90

```
def hit_rate_at_k(results: list, k: int = 5) -> float:
    hits = sum(
        1 for r in results
        if r["expected_pattern_ids"]
        and any(
            chunk["metadata"]["pattern_id"] in r["expected_pattern_ids"]
            for chunk in r["retrieved_chunks"][:k]
        )
    )
    positives = sum(1 for r in results if r["expected_pattern_ids"])
    return hits / positives if positives else 0.0
```

```
def mrr_at_k(results: list, k: int = 5) -> float:
    reciprocal_ranks = []
    for r in results:
        if not r["expected_pattern_ids"]:
            continue
        for rank, chunk in enumerate(r["retrieved_chunks"][:k], start=1):
            if chunk["metadata"]["pattern_id"] in r["expected_pattern_ids"]:
                reciprocal_ranks.append(1 / rank)
                break
        else:
            reciprocal_ranks.append(0.0)
    return sum(reciprocal_ranks) / len(reciprocal_ranks) if reciprocal_ranks else 0.0
```

Generation Metrics — RAGAS (LLM-assisted)

Metric	Definition	Target
Faithfulness	Answer claims supported by retrieved context	≥ 0.90
Answer Relevance	Semantic alignment between answer and query	≥ 0.85
Context Precision	Retrieved chunks actually used in the answer	≥ 0.70
Context Recall	Ground truth coverage by retrieved context	≥ 0.80
Answer Correctness	Semantic similarity vs ground truth answer	≥ 0.75

```

from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall, answer_correctness,
)
from datasets import Dataset

def run_ragas_evaluation(test_cases: list[dict]) -> dict:
    data = {
        "question": [tc["query"] for tc in test_cases],
        "answer": [tc["generated_answer"] for tc in test_cases],
        "contexts": [tc["retrieved_texts"] for tc in test_cases],
        "ground_truth": [tc["ground_truth_answer"] for tc in test_cases]
    }
    return evaluate(Dataset.from_dict(data), metrics=[
        faithfulness, answer_relevancy,
        context_precision, context_recall, answer_correctness,
    ])

```

System-Specific Metrics

Metric	Definition	Target
Citation Accuracy	Answers citing the correct pattern_id and section	≥ 0.90
Negative Handling Rate	Negative queries answered with an explicit "no runbook found"	$= 1.00$
Hallucination Rate	Answer claims not traceable to any retrieved chunk	≤ 0.05

```

import re

def citation_accuracy(test_cases: list[dict]) -> float:
    correct, total = 0, 0
    for tc in test_cases:
        if not tc["expected_pattern_ids"]:
            continue
        cited = re.findall(r"Pattern_\d+", tc["generated_answer"])
        if any(p in tc["expected_pattern_ids"] for p in cited):
            correct += 1
        total += 1
    return correct / total if total else 0.0

def negative_handling_rate(test_cases: list[dict]) -> float:
    negatives = [tc for tc in test_cases if not tc["expected_pattern_ids"]]
    handled = sum(
        1 for tc in negatives
        if "no relevant runbook" in tc["generated_answer"].lower()
        or "not covered" in tc["generated_answer"].lower()
        or "cannot find" in tc["generated_answer"].lower()
    )
    return handled / len(negatives) if negatives else 0.0

```

7.4 Evaluation Pipeline

Script Layout

eval/	
├─ golden_test_set.csv	← 57 test cases (source of truth)
├─ run_retrieval_eval.py	← retrieval metrics only – no LLM, fast
├─ run_generation_eval.py	← RAGAS + system-specific metrics – slower
├─ report.py	← prints comparison table, writes to report
└─ results/	← local JSON snapshots per run
├─ phase2_baseline.json	
└─ phase3_hybrid.json	

Run Commands

```

# Step 1 – retrieval only (fast, no LLM cost)
python eval/run_retrieval_eval.py --phase phase2_baseline --k 5

# Step 2 – full generation eval (~$0.50-$2 per run)
python eval/run_generation_eval.py --phase phase2_baseline

# Step 3 – compare two phases side by side
python eval/report.py --baseline phase2_baseline --compare phase3_hy

```

Results Storage in Supabase

```

create table eval_runs (
  id          bigserial primary key,
  phase       text          not null,    -- e.g. 'phase2_baseline', 'pl
  run_at      timestamptz default now(),
  retrieval   jsonb,         -- HR@5, MRR@5, precision, rec
  generation  jsonb,         -- RAGAS metrics
  system      jsonb,         -- citation_accuracy, negative
  notes       text
);

```

7.5 Phase Gate Criteria

A gate failure blocks progression to the next phase. It does not trigger automatic rollback but requires investigation before moving forward.

Gate A — After Phase 2 (Baseline)

Metric	Minimum to Pass
HR@5	≥ 0.70
Negative Handling Rate	= 1.00 (non-negotiable)
Faithfulness	≥ 0.85

Gate B — After Phase 3 (Hybrid Retrieval)

Metric	Minimum to Pass	Must Improve vs Gate A
HR@5	≥ 0.85	Yes
Error Code Routing Accuracy	≥ 0.90	Yes — primary motivator for Supabase FTS

Context Precision @ 5	≥ 0.70	—
Faithfulness	≥ 0.90	—
Negative Handling Rate	$= 1.00$	Maintained

Gate C — After Phase 6 (Production)

Metric	Minimum to Pass
HR@5	≥ 0.85
MRR@5	≥ 0.75
Faithfulness	≥ 0.90
Answer Correctness	≥ 0.75
Citation Accuracy	≥ 0.90
Negative Handling Rate	$= 1.00$
Hallucination Rate	≤ 0.05

7.6 Metric Targets at a Glance

Metric	Type	Phase 2	Phase 3	Phase 6
Hit Rate @ 5	Retrieval	≥ 0.70	≥ 0.85	≥ 0.85
MRR @ 5	Retrieval	—	≥ 0.75	≥ 0.75
Context Precision @ 5	Retrieval	—	≥ 0.70	≥ 0.70
Context Recall	Retrieval	—	≥ 0.80	≥ 0.80
Error Code Routing	Retrieval	—	≥ 0.90	≥ 0.90
Escalation Routing	Retrieval	—	≥ 0.90	≥ 0.90
Faithfulness	Generation	≥ 0.85	≥ 0.90	≥ 0.90
Answer Relevance	Generation	—	—	≥ 0.85
Answer Correctness	Generation	—	—	≥ 0.75
Citation Accuracy	System	—	—	≥ 0.90
Negative Handling Rate	System	$= 1.00$	$= 1.00$	$= 1.00$
Hallucination Rate	System	—	—	≤ 0.05

Dependencies

Production (requirements.txt)

```
# Core
fastapi
uvicorn # local dev only; Vercel uses its own ASGI server

# LangChain
langchain
langchain-community
langchain-openai

# Vector store + database
supabase

# Document parsing
markdown-it-py

# API
httpx
pydantic

# Utilities
python-dotenv # local dev only
```

Eval Only (not deployed to Vercel)

```
ragas>=0.1.0
datasets
pandas
```

Phase Summary

Phase	Focus	Key Decisions
1	Ingestion Pipeline	markdown-it-py parser; Supabase pgvector; ###/### chunking; 12-field metadata; header injection; Sections 3/8/9 excluded
2	Core RAG (Single Turn)	SupabaseVectorStore retriever; LLM abstraction layer; FastAPI /api/analyze

3	Hybrid Retrieval	Dense (pgvector) + Sparse (Supabase FTS) + RRF; HyDE; metadata pre-filtering
4	Splunk Log Integration	Log summarizer; structured summary injected into retrieval context
5	Multi-Turn Conversation	Stateless API; client-side history window; history-aware query reformulation
6	Hardening & Production	Vercel deployment; vercel.json; Vercel env vars; Pro plan required
7	RAG Evaluation	57-case golden test set; RAGAS + retrieval metrics; 3 phase gates (A/B/C)